

```
In [9]: import pandas as pd
```

```
In [10]: pd.__version__
```

```
Out[10]: '2.3.1'
```

```
In [11]: !pip install openpyxl
```

Requirement already satisfied: openpyxl in c:\users\ipl4\anaconda3\lib\site-packages (3.1.5)
Requirement already satisfied: et-xmlfile in c:\users\ipl4\anaconda3\lib\site-packages (from openpyxl) (2.0.0)

```
In [12]: emp=pd.read_excel(r"C:\Users\IPL4\Desktop\work\Rawdata.xlsx")
```

```
In [6]: emp
```

```
Out[6]:
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#\$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1\$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^\$0	10+

```
In [18]: print(emp.head()) #first 5 rows of data frame
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#\$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1\$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year

```
In [15]: id(emp) #address
```

```
Out[15]: 2247658649872
```

```
In [16]: emp.columns #columns names
```

```
Out[16]: Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

```
In [17]: emp.shape #row and column
```

```
Out[17]: (6, 6)
```

```
In [19]: emp.tail() # Last 5 rows of the DataFrame
```

Out[19]:

	Name	Domain	Age	Location	Salary	Exp
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1\$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^\$0	10+

In [20]: `emp.info()` *#summary of dataframe*

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Name        6 non-null      object
1   Domain       6 non-null      object
2   Age          4 non-null      object
3   Location     4 non-null      object
4   Salary       6 non-null      object
5   Exp         5 non-null      object
dtypes: object(6)
memory usage: 416.0+ bytes
```

In [21]: `emp`

Out[21]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#\$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1\$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^\$0	10+

In [23]: `emp.isnull()` *##check for missing values*

Out[23]:

	Name	Domain	Age	Location	Salary	Exp
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	True	True	False	False
3	False	False	True	False	False	True
4	False	False	False	True	False	False
5	False	False	False	False	False	False

```
In [24]: emp.isna() ##check for missing values
```

```
Out[24]:
```

	Name	Domain	Age	Location	Salary	Exp
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	True	True	False	False
3	False	False	True	False	False	True
4	False	False	False	True	False	False
5	False	False	False	False	False	False

```
In [26]: emp.isnull().sum()#total null values in each column
```

```
Out[26]: Name      0
Domain    0
Age       2
Location  2
Salary    0
Exp       1
dtype: int64
```

```
In [27]: emp['Name'] # extracting column
```

```
Out[27]: 0      Mike
1      Teddy^
2      Uma#r
3      Jane
4      Uttam*
5      Kim
Name: Name, dtype: object
```

```
In [34]: emp['Name'] = emp['Name'].str.replace(r'\W', '', regex=True)
```

emp['Name'] → selects the Name column.

.str.replace(pattern, replacement, regex=True) → applies string replacement on each value.

r'\W' → is a regular expression (regex) that means:

\W → matches any non-word character (anything except A–Z, a–z, 0–9, and _).

r before the string makes it a raw string (so \W is interpreted correctly).

" → means replace with nothing (i.e., remove it).

regex=True → tells Pandas to treat the first argument as a regex pattern.

```
In [35]: emp['Name']
```

```
Out[35]: 0    Mike
         1    Teddy
         2    Umar
         3    Jane
         4    Uttam
         5    Kim
         Name: Name, dtype: object
```

```
In [36]: emp['Domain']
```

```
Out[36]: 0    Datascience#$
         1    Testing
         2    Dataanalyst^^#
         3    Ana^alytics
         4    Statistics
         5    NLP
         Name: Domain, dtype: object
```

```
In [37]: emp['Domain'] = emp['Domain'].str.replace(r'\W', '', regex=True)
```

```
In [38]: emp['Domain']
```

```
Out[38]: 0    Datascience
         1    Testing
         2    Dataanalyst
         3    Analytics
         4    Statistics
         5    NLP
         Name: Domain, dtype: object
```

```
In [39]: emp['Age']
```

```
Out[39]: 0    34 years
         1    45' yr
         2    NaN
         3    NaN
         4    67-yr
         5    55yr
         Name: Age, dtype: object
```

```
In [40]: emp['Age'] = emp['Age'].str.replace(r'\W', '', regex=True)
```

```
In [41]: emp['Age']
```

```
Out[41]: 0    34years
         1    45yr
         2    NaN
         3    NaN
         4    67yr
         5    55yr
         Name: Age, dtype: object
```

```
In [42]: emp['Age'] = emp['Age'].str.extract('(\d+)')
```

emp['Age'] → selects the Age column.

.str.extract('(\d+)') → extracts part of the string that matches the regex pattern. Regex

Explanation:

\d → means digit (0–9).

- → means one or more digits.

(\d+) → the parentheses () create a capturing group to extract those digits.

```
In [43]: emp['Age']
```

```
Out[43]: 0      34
          1      45
          2     NaN
          3     NaN
          4      67
          5      55
          Name: Age, dtype: object
```

```
In [44]: emp['Location'] = emp['Location'].str.replace(r'\W', '', regex=True)
```

```
In [45]: emp['Location']
```

```
Out[45]: 0      Mumbai
          1    Bangalore
          2         NaN
          3    Hyderbad
          4         NaN
          5      Delhi
          Name: Location, dtype: object
```

```
In [46]: emp
```

```
Out[46]:
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5^00#0	2+
1	Teddy	Testing	45	Bangalore	10%%000	<3
2	Umar	Dataanalyst	NaN	NaN	1\$5%000	4> yrs
3	Jane	Analytics	NaN	Hyderbad	2000^0	NaN
4	Uttam	Statistics	67	NaN	30000-	5+ year
5	Kim	NLP	55	Delhi	6000^\$0	10+

```
In [47]: emp['Salary']
```

```
Out[47]: 0      5^00#0
          1     10%%000
          2     1$5%000
          3      2000^0
          4      30000-
          5      6000^$0
          Name: Salary, dtype: object
```

```
In [48]: emp['Salary'] = emp['Salary'].str.replace(r'\W', '', regex=True)
```

```
In [49]: emp['Salary']
```

```
Out[49]: 0      5000
         1     10000
         2     15000
         3     20000
         4     30000
         5     60000
         Name: Salary, dtype: object
```

```
In [50]: emp['Exp']=emp['Exp'].str.extract('(\d+)')
```

```
In [51]: emp['Exp']
```

```
Out[51]: 0      2
         1      3
         2      4
         3    NaN
         4      5
         5     10
         Name: Exp, dtype: object
```

```
In [52]: clean_data=emp.copy()
```

```
In [53]: clean_data
```

```
Out[53]:
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	NaN	NaN	15000	4
3	Jane	Analytics	NaN	Hyderbad	20000	NaN
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

```
In [55]: import numpy as np
```

```
In [56]: clean_data['Age'] = clean_data['Age'].fillna(np.mean(pd.to_numeric(clean_data['A
```

`clean_data['Age']` Selects the Age column from your DataFrame.

`pd.to_numeric(clean_data['Age'])` Converts the Age column to numeric (int/float).

If the column contains strings (like "23"), it will convert them to numbers.

`np.mean(...)` Calculates the mean (average) of the numeric Age values.

`.fillna(value)` Replaces all NaN (missing values) in the Age column with the given value.

Here, the value is the mean age. `clean_data['Age'] = ...` Stores the cleaned column back into the DataFrame.

```
In [57]: clean_data['Age']
```

```
Out[57]: 0      34
         1      45
         2    50.25
         3    50.25
         4      67
         5      55
         Name: Age, dtype: object
```

```
In [58]: clean_data['Exp'] = clean_data['Exp'].fillna(np.mean(pd.to_numeric(clean_data['E
```

```
In [59]: clean_data['Exp']
```

```
Out[59]: 0      2
         1      3
         2      4
         3    4.8
         4      5
         5     10
         Name: Exp, dtype: object
```

```
In [61]: print(clean_data['Location'].isnull().sum())
```

```
2
```

```
In [62]: clean_data['Location'] = clean_data['Location'].fillna(clean_data['Location'].mo
```

```
In [63]: clean_data['Location']
```

```
Out[63]: 0      Mumbai
         1    Bangalore
         2    Bangalore
         3    Hyderbad
         4    Bangalore
         5      Delhi
         Name: Location, dtype: object
```

```
In [64]: clean_data.info() # Displaying information about the cleaned DataFrame
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Name        6 non-null      object
1   Domain      6 non-null      object
2   Age         6 non-null      object
3   Location    6 non-null      object
4   Salary      6 non-null      object
5   Exp         6 non-null      object
dtypes: object(6)
memory usage: 416.0+ bytes
```

```
In [65]: clean_data['Age']=clean_data['Age'].astype(int) # Converting 'Age' column to int
```

```
In [66]: clean_data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Name        6 non-null      object
 1   Domain      6 non-null      object
 2   Age         6 non-null      int64
 3   Location    6 non-null      object
 4   Salary      6 non-null      object
 5   Exp         6 non-null      object
dtypes: int64(1), object(5)
memory usage: 416.0+ bytes

```

```
In [67]: clean_data['Salary']=clean_data['Salary'].astype(int)
```

```
In [68]: clean_data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Name        6 non-null      object
 1   Domain      6 non-null      object
 2   Age         6 non-null      int64
 3   Location    6 non-null      object
 4   Salary      6 non-null      int64
 5   Exp         6 non-null      object
dtypes: int64(2), object(4)
memory usage: 416.0+ bytes

```

```
In [69]: clean_data['Exp']=clean_data['Exp'].astype(int)
```

```
In [70]: clean_data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Name        6 non-null      object
 1   Domain      6 non-null      object
 2   Age         6 non-null      int64
 3   Location    6 non-null      object
 4   Salary      6 non-null      int64
 5   Exp         6 non-null      int64
dtypes: int64(3), object(3)
memory usage: 416.0+ bytes

```

```
In [71]: clean_data['Name'] = clean_data['Name'].astype('category') # Converting 'Name' c
clean_data['Domain'] = clean_data['Domain'].astype('category') # Converting 'Nam
clean_data['Location'] = clean_data['Location'].astype('category') # Converting
```

```
In [72]: clean_data.info()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Name        6 non-null     category
1   Domain      6 non-null     category
2   Age         6 non-null     int64
3   Location    6 non-null     category
4   Salary      6 non-null     int64
5   Exp         6 non-null     int64
dtypes: category(3), int64(3)
memory usage: 934.0 bytes
```

```
In [74]: clean_data.to_csv('clean_data.csv') #saving clean data frame to csv file
```

```
In [75]: import os #importing os module for opertaing system dependent functionality
```

```
In [76]: os.getcwd() #get thecurrent working directory
```

```
Out[76]: 'C:\\\\Users\\IPL4'
```

```
In [ ]:
```